

Day-74-Loading and Inspecting Data with Pandas

1. Day Number

Day 74

2. Topic Name

Reading CSV, Excel, and JSON into Pandas DataFrames

Today you will learn how Pandas can load different file formats and let you inspect them using the same **DataFrame** interface.

3. Connection

You already learned how to work with:

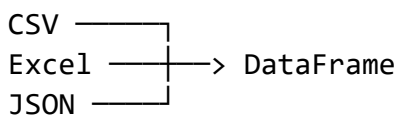
- CSV files
- Excel files
- JSON files
- Pandas DataFrames

Previously, each file type needed its own way of reading data.

Today, Pandas gives you one common workflow:

File → Pandas DataFrame → Inspect the data

For example:



Once the data becomes a DataFrame, you can inspect it in similar ways regardless of where it came from.

4. Important Topics

`read_csv()`

Used to load a CSV file into a DataFrame.

General idea:

```
pd.read_csv("filename.csv")
```

`read_excel()`

Used to load an Excel worksheet into a DataFrame.

General idea:

```
pd.read_excel("filename.xlsx")
```

Pandas normally uses an Excel-reading package such as openpyxl for .xlsx files.

JSON loading idea

Pandas can also read JSON data.

General idea:

```
pd.read_json("filename.json")
```

The exact structure of the JSON matters because JSON does not always contain simple rows and columns.

head()

Shows the **first few rows** of a DataFrame.

```
data.head()
```

By default, it usually displays the first **5 rows**.

shape

Shows the number of:

```
(rows, columns)
```

Example:

```
(5, 3)
```

means:

```
5 rows  
3 columns
```

Notice that shape is an **attribute**, so we do not write parentheses after it.

columns

Shows the column names.

For example:

```
product  
price
```

```
stock
```

dtypes

Shows the data type Pandas detected for every column.

Possible examples include:

```
int64  
float64  
object
```

For now, think of object as something commonly used for text columns.

5. Foundational Notes

When working with real data, loading the file is only the beginning.

A simple Pandas workflow is:

1. Import Pandas
2. Load the dataset
3. Inspect the first rows
4. Check its size
5. Check column names
6. Check data types
7. Only then decide what needs to be changed

Suppose you have:

```
products.csv
```

with:

```
product,price,stock  
Laptop,55000,5  
Mouse,700,20  
Keyboard,1500,12
```

Pandas can convert this file into something conceptually like:

	product	price	stock
0	Laptop	55000	5
1	Mouse	700	20
2	Keyboard	1500	12

This is a **DataFrame**.

6. Why Inspect Data Before Changing It?

Imagine receiving a box without looking inside and immediately trying to reorganize everything.

You could easily make mistakes.

Data is similar.

Before changing a dataset, you should first understand:

```
What columns exist?  
How many rows are there?  
What does the data look like?  
What data types were detected?  
Are there unexpected columns?
```

For example, you might expect:

```
product  
price  
stock
```

but discover the actual file contains:

```
product  
price  
stock  
supplier
```

If you start modifying the dataset without inspecting it, your program may make incorrect assumptions.

A good beginner habit is:

| Load first. Inspect second. Change later.

Today we focus only on loading and inspection.

7. Easy Example

Imagine a CSV file named:

```
students.csv
```

containing:

```
name,score  
Asha,85
```

```
Ravi,92  
Neha,78
```

After loading it into Pandas, you might inspect it with ideas such as:

```
students.head()  
students.shape  
students.columns  
students.dtypes
```

Conceptually, you would discover:

```
First rows:  
Asha      85  
Ravi      92  
Neha      78  
  
Shape:  
(3, 2)  
  
Columns:  
name  
score  
  
Data types:  
name      text-like  
score     integer
```

You are not changing anything yet.

You are simply learning about the dataset.

8. Problem Statement

Create a small CSV dataset containing information about products.

Use columns such as:

```
product  
price  
stock
```

Example file content:

```
product,price,stock  
Laptop,55000,5  
Mouse,700,20  
Keyboard,1500,12
```

Monitor,12000,8 Webcam,2500,0

Your program should:

1. Load the CSV file into a Pandas DataFrame.
2. Display the first few rows.
3. Display the DataFrame's dimensions.
4. Display its column names.
5. Display the data type of each column.

Do **not** clean, delete, rename, or modify columns yet.

The purpose is only to **load and inspect**.

9. Concepts Used

You will practice:

- importing Pandas
- DataFrames
- loading CSV files
- `read_csv()`
- understanding `read_excel()` conceptually
- understanding JSON loading conceptually
- `head()`
- `shape`
- `columns`
- `dtypes`
- rows and columns
- basic dataset inspection

A useful distinction is:

Method: <code>head()</code>

Attributes: <code>shape</code> <code>columns</code> <code>dtypes</code>
--

So:

<code>data.head()</code>

uses parentheses.

But:

```
data.shape
```

does not.

10. Thought Process

Before writing the program, think through the problem like this:

Step 1: What file am I loading?

```
products.csv
```

Step 2: Which Pandas function can read CSV files?

```
read_csv()
```

Step 3: Where should the loaded data be stored?

Inside a DataFrame variable.

Conceptually:

```
dataframe = load CSV file
```

Step 4: What should I inspect first?

The first few rows.

Think:

```
DataFrame → head()
```

Step 5: How large is the dataset?

Check:

```
shape
```

Step 6: What columns exist?

Check:

```
columns
```

Step 7: What type of data is stored in each column?

Check:

dtypes

The overall thinking is:

```
Load
↓
Preview
↓
Check dimensions
↓
Check columns
↓
Check data types
```

11. Pseudocode

```
START

Import Pandas

Load products.csv into a DataFrame

Display the first few rows

Display the shape of the DataFrame

Display the column names

Display the data types of each column

END
```

A slightly more detailed version:

```
START

Import Pandas using its common short name

Create a DataFrame by reading products.csv

Call the method that displays the first rows

Check the attribute containing rows and columns count

Check the attribute containing column names
```

Check the attribute containing each column's data type

END

12. Suggested Solving Approach: Pandas Approach

Use one DataFrame throughout the exercise.

Your mental model should be:

```
products.csv
  ↓
pd.read_csv(...)
  ↓
DataFrame
  ↓
Inspect it
```

Then investigate the DataFrame with:

```
head()
shape
columns
dtypes
```

You do not need loops for this exercise.

You also do not need to manually open the CSV using Python's `csv` module because today's goal is specifically to practice the **Pandas approach**.

For other formats, remember the same idea:

```
CSV    → read_csv()    → DataFrame
Excel  → read_excel()  → DataFrame
JSON   → read_json()   → DataFrame
```

The loading function changes, but once the result is a DataFrame, many inspection tools stay the same.

13. Easy Edge Cases

Empty dataset

Imagine:

```
product,price,stock
```

There are column names but no product rows.

Your DataFrame may have a shape similar to:

```
(0, 3)
```

Meaning:

```
0 rows  
3 columns
```

`head()` would have no actual data rows to display.

Unexpected column

Suppose the file contains:

```
product,price,stock,supplier  
Laptop,55000,5,ABC Ltd
```

You expected three columns, but the dataset contains four.

Checking:

```
columns
```

helps you discover this before changing the data.

This demonstrates why inspection is important.

14. Expected Output

For a dataset such as:

```
product,price,stock  
Laptop,55000,5  
Mouse,700,20  
Keyboard,1500,12  
Monitor,12000,8  
Webcam,2500,0
```

your output should conceptually look similar to:

First rows:

	product	price	stock
0	Laptop	55000	5
1	Mouse	700	20

2	Keyboard	1500	12
3	Monitor	12000	8
4	Webcam	2500	0

Shape:

(5, 3)

Columns should represent something similar to:

product
price
stock

Data types should be approximately:

product	object
price	int64
stock	int64

The exact integer type can occasionally vary depending on the environment and dataset.

15. Hint Only

Start with the normal Pandas import:

```
import pandas as pd
```

Then think about which Pandas function matches a **CSV file**.

Store the resulting DataFrame in a variable such as:

products

Finally, remember this difference:

First rows	→ head()
Dimensions	→ shape
Column names	→ columns
Data types	→ dtypes

Your program only needs to **load and inspect** the data. Do not perform cleaning or transformations yet.

Chat/file name: Day-74-Loading and Inspecting Data with Pandas